Master Document

Open Source Project as Inspiration and Foundation:

Sending Encrypted Messages with a Simple Chat Program in Python – Python Garden

Other links:

How to install Ubuntu Server on your Raspberry Pi | Ubuntu

TASKS

MILESTONE 1 – ENVIRONMENT SETUP

To get all Pi's ready to run Python Garden chat system, use the following code:

1. To update each PI:

```
sudo apt update && sudo apt upgrade -y
```

2. To install required system packages:

```
sudo apt install build-essential libssl-dev python3-dev  -y
```

3. To confirm python and pip are installed:

```
python3 –version

pip3 –version
```

4. To check if cryptography is present (will get requirement already satisfied if present):

```
python3 -m pip install –user –break-system-packages
cryptography
```

5. Verify the cryptography version:

```
python3 -c "import cryptography;
print(cryptography.__version__)"
```

6. To create a dedicated project directory:

```
mkdir ~/python_garden_chat
```

7. Move into the project directy:

```
cd ~ /python_garden_chat
```

MILESTONE 2 – BASE PYTHON GARDEN CODE RUNNING

To get the core chat system running on a single pi before expanding it to multiple Pi's:

1. Create three core project files inside ~/python_garden_chat directory:

```
touch server.py client.py encryption.py
```

2. Implement the encryption module:

2.1: Open encryption.py

```
nano encryption.py
```

2.2: Paste the following code, then use ctrl+x and click y, then enter to exit

```
from cryptography.fernet import Fernet

import os


KEY_FILE = "secret.key"


def load_key():

    """Load the encryption key from a file, or generate one if it
    doesn't  exist.

    Returns a Fernet key."""

    if os.path.exists(KEY_FILE):

        with open(KEY_FILE, "rb") as f:

        key = f.read()

    else:

        key = Fernet.generate_key()

        with open(KEY_FILE, "wb") as f:

            f.write(key)
```

```
        return key

    def encrypt_message(key, message):

        """Encrypt a plaintext message (string) and return encrypted
    bytes."""

        f = Fernet(key)

        return f.encrypt(message.encode())

    def decrypt_message(key, token):

        """Decrypt encrypted bytes and return the plaintext string."""

        f = Fernet(key)

        return f.decrypt(token).decode()
```

2.3: Local test (run in terminal) that should output a key, encrypted and decrypted message:

```
python3 -c "from encryption import load_key, encrypt_message,
decrypt_message; k=load_key(); e=encrypt_message(k,'hello');
print('Encrypted:',e); print('Decrypted:',decrypt_message(k,e))"
```

3. Implement the server

3.1: Open server.py and add starter structure:

```
nano server.py
```

3.2: Paste the following code, then use ctrl+x and click y, then enter to exit

```
import socket
import threading
import time
from cryptography.fernet import Fernet

SERVER_NAME = "SERVER"

def load_key():
    """ Load the shared Fernet encryption key """
    return open("secret.key", "rb").read()

def encrypt_message(message, key):
    """ Encrypt a plaintext message using Fernet """
    return Fernet(key).encrypt(message.encode())

def decrypt_message(message, key):
```

```
    """ Decrypt a Fernet-encrypted message """
    return Fernet(key).decrypt(message).decode()

class Server:
    def __init__(self, host="0.0.0.0", port=5555):
        self.host = host
        self.port = port
        self.key = load_key()

        self.server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.server.bind((self.host, self.port))
        self.server.listen()

        # Allow clean shutdown by timing out accept()
        self.server.settimeout(1)

        self.clients = []
        self.nicknames = []
        self.addresses = []
        self.running = True

        # Lock prevents printing collisions
        self.print_lock = threading.Lock()

        print("[SERVER STARTED]")
        print(f"[LISTENING] Server running on {self.host}:{self.port}\n")
        print("Menu Option: ")
        print("Type /quit to shut down the server.")
        print("Type /users to view everyone and their IP address in the chat.")
        print("Type /whoami to view Self and IP address.")
        print("Hit enter to bring prompt back up!\n")

    def broadcast(self, message, sender=None):
        """Send a plaintext message to all clients (encrypted).
           Sender is excluded so messages are not echoed back."""

        encrypted = encrypt_message(message, self.key)

        for client in self.clients:
            if client != sender:
                try:
                    client.send(encrypted)
                except:
                    pass

    def handle_client(self, client):
        """Handle incoming messages from a single client"""
        while self.running:
            try:
```

```
                encrypted_data = client.recv(4096)

                if not encrypted_data:
                    break

                decrypted = decrypt_message(encrypted_data, self.key)

                # command handling
                if decrypted.startswith("/users"):
                    user_list = "\n".join(f" - {name}" for name in
self.nicknames)

                    response = (
                        f"[SERVER]: Users Online ({len(self.nicknames)})\n"
                        f"{user_list}"
                    )

                    encrypted_response = encrypt_message(response, self.key)

                    try:
                        client.send(encrypted_response)
                    except:
                        pass

                    continue

                # /whoami command
                if decrypted.startswith("/whoami"):
                    index = self.clients.index(client)
                    nickname = self.nicknames[index]
                    addr = self.addresses[index]

                    response = f"[SERVER]: You are {nickname} ({addr})"

                    client.send(encrypt_message(response, self.key))
                    continue

                # relay message to other clients
                encrypted_out = encrypt_message(decrypted, self.key)

                for c in self.clients:
                    if c != client:
                        try:
                            c.send(encrypted_out)
                        except:
                            pass

                # print once on server
                if not decrypted.startswith(f"[{SERVER_NAME}]"):
```

```
                    with self.print_lock:
                        print(f"\r{decrypted}")
                        print(f"\r[{SERVER_NAME}]: ", end="", flush=True)

            except:
                break

        # Handle disconnect
        self.remove_client(client)

    def remove_client(self, client):
        """Remove a client cleanly when they disconnect"""
        if client in self.clients:
            index = self.clients.index(client)
            nickname = self.nicknames[index]

            self.clients.remove(client)
            self.nicknames.pop(index)
            self.addresses.pop(index)

            try:
                client.close()
            except:
                pass

            leave_msg = f"[LEAVE] {nickname}"

            with self.print_lock:
                print(f"\r{leave_msg}")
                print(f"[{SERVER_NAME}]: ", end="", flush=True)

    def write_loop(self):
        """Allow the server operator to send messages."""
        while self.running:
            text = input(f"[{SERVER_NAME}]: ").strip()

            if not text:
                continue

            # server command: list connected users
            if text == "/users":
                with self.print_lock:
                    if self.nicknames:
                        print(f"[SERVER]: Connected users
({len(self.nicknames)}):")
                        for name, addr in zip(self.nicknames, self.addresses):
                            print(f" - {name} ({addr[0]}:{addr[1]})")
                    else:
                        print("[SERVER]: No users connected.")
```

```
                continue

            # server command: /whoami
            if text == "/whoami":
                with self.print_lock:
                    print(f"\r[SERVER]: You are {SERVER_NAME}
({self.host}:{self.port})")
                continue

            # server command: /quit
            if text.lower() == "/quit":
                shutdown_msg = "[SERVER]: Server is shutting down."

                with self.print_lock:
                    print(shutdown_msg)

                self.broadcast(shutdown_msg)
                self.shutdown()
                break

            msg = f"[{SERVER_NAME}]: {text}"

            # send only to clients
            encrypted = encrypt_message(msg, self.key)

            for client in self.clients:
                try:
                    client.send(encrypted)
                except:
                    pass

    def shutdown(self):
        """ Cleanly shut down the server and all client connections"""

        self.running = False

        # Close all client sockets
        for client in self.clients:
            try:
                client.shutdown(socket.SHUT_RDWR)
                client.close()
            except:
                pass

        try:
            self.server.close()
        except:
            pass
```

```
        print("[SERVER SHUTDOWN] Server stopped cleanly.")

    def receive(self):
        """ Accept new clients and perform nickname handshake"""
        while self.running:
            try:
                client, address = self.server.accept()
            except socket.timeout:
                continue
            except OSError:
                break

            # Request nickname
            client.send(b"NICK")
            nickname = client.recv(4096).decode("ascii")

            self.clients.append(client)
            self.nicknames.append(nickname)
            self.addresses.append(address)

            join_msg = f"[JOIN] {nickname}"

            with self.print_lock:
                print(f"\r{join_msg}\n")
                print(f"[{SERVER_NAME}]: ", end="", flush=True)

            self.broadcast(join_msg)

            # small delay ensures JOIN arrives before user messages
            time.sleep(0.05)

            # Start thread for this client
            thread = threading.Thread(target=self.handle_client,
args=(client,))
            thread.start()

# Start the server
if __name__ == "__main__":
    server = Server()
    threading.Thread(target=server.receive, daemon=True).start()
    server.write_loop()
```

4. Implement the client:

4.1: Open client.py:

```
nano client.py
```

4.2: Paste the following code, then use ctrl+x and click y, then enter to exit

```python
import socket

import threading

from cryptography.fernet import Fernet

def load_key():
    """ Load the shared Fernet encryption key """
    return open("secret.key", "rb").read()

def encrypt_message(message, key):
    """ Encrypt a plaintext message using Fernet """
    return Fernet(key).encrypt(message.encode())

def decrypt_message(message, key):
    """ Decrypt a Fernet-encrypted message """.
    return Fernet(key).decrypt(message).decode()

class Client:
    def __init__(self, host, port=5555):
        self.nickname = input("Enter nickname: ")
        self.key = load_key()

        self.client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.client.connect((host, port))

        self.print_lock = threading.Lock()
        self.ready = threading.Event()
        self.typing = False

        # Handshake: Server requests nickname
        first_msg = self.client.recv(4096)
        if first_msg == b"NICK":
            self.client.send(self.nickname.encode('ascii'))

        print("\nConnected to chat.\n")
        print("Menu Options: ")
        print("Type /quit to leave.")
        print("Type /users to view others in the chat.")
        print("Type /whoami to view your Nickname and your IP address.")
        print("Hit enter to bring the prompt back!\n")

        # Start background receive thread
        threading.Thread(target=self.receive, daemon=True).start()

        # Wait for first server message
        self.ready.wait()

        # Main thread handles user input
        self.write_loop()
```

```
def receive(self):
    """ Continuously receive and decrypt messages from the server """
    while True:
        try:
            message = self.client.recv(4096)

            # Ignore empty packets
            if not message:
                break

            decrypted = decrypt_message(message, self.key)

            with self.print_lock:

                if self.typing:
                    # move to start and clear line
                    print("\r\033[K", end="")

                # print incoming message
                print(decrypted)

                if self.typing:
                    print(f"[{self.nickname}]: ", end="", flush=True)

            self.ready.set()

        except:
            print("\n[DISCONNECTED] Server has shut down.")
            break

def write_loop(self):
    """ Main thread: read user input and send encrypted messages """

    while True:
        try:
            self.typing = True
            text = input(f"[{self.nickname}]: ").strip()
            self.typing = False

            # normalize immediately
            text = text.strip()

            # hard block empty input
            if text == "":
                continue

            # Clean exit command
            if text.lower() == "/quit":
```

```
                try:
                    self.client.close()
                except:
                    pass
                print("You have disconnected from the chat.")
                break

            # command vs normal message
            if text.startswith("/"):
                msg = text
            else:
                msg = f"[{self.nickname}]: {text}"

            if not msg.strip():
                continue

            encrypted = encrypt_message(msg, self.key)
            self.client.send(encrypted)

        except:
            print("[DISCONNECTED] Server has shut down.")
            break

host = input("Enter server ZeroTier IP: ")
Client(host)
```

5. Run a full localhost test:

5.1: In terminal 1, start the server (should see [server starting] and [listening]):

```
python3 server.py
```

5.2: In terminal 2, start the client (should see [client starting] and [connected]):

```
python3 client.py
```

Total code versions: 4 total versions

Working and finalized version: 4th version

MILESTONE 3 – MULTIPLE PI NETWORKING

Create the ZeroTier VPN

1. Run the following to install the ZeroTier client service:

```
curl -s https://install.zerotier.com | sudo bash
```

    1.1: If curl not downloaded:

        1.1.1: sudo apt update

        1.1.2: sudo apt install -y curl

        1.1.3: curl --version

2. Create the ZeroTier virtual network (server pi)

3. Use the following network ID to connect pi to the network

```
Network ID: b103a835d24e1dd8
```

4. On each pi, run the following:

```
sudo zerotier-cli join b103a835d24e1dd8
```

5. Server Pi should approve each pi and grant a VLAN IP for python garden use

```
Nyssa's ZeroTier IPv4: 10.178.192.58/24

Luke ZeroTier IPv4: 10.178.192.63/24

Sebastian ZeroTier IPv4: 10.178.192.165/24
```

6. Server Pi runs zerotier-cli listnetworks ((assignedAddresses field is the zerotier ip))

7. Server Pi runs python3 server.py while Client Pis run python3 client.py:

    Client.py needs to use the server's ZT IP address when prompted: 10.178.192.58

8. Clients assign themselves a nickname after successfully joining ZeroTier Network

Screenshots from Group Meetings (no connectivity)

```
(venv) lukes@lab-pi:~/encrypted_chat_client$ python client.py
Enter server ZeroTier IP: 10.178.192.58
Enter nickname: Luke
Traceback (most recent call last):
  File "/home/lukes/encrypted_chat_client/client.py", line 49, in <module>
    Client(host).run()
    ~~~~~~~~^^^^^^
  File "/home/lukes/encrypted_chat_client/client.py", line 19, in __init__
    self.client.connect((host, port))
    ~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^
TimeoutError: [Errno 110] Connection timed out
```

```
(venv) lukes@lab-pi:~/encrypted_chat_client$ deactivate
lukes@lab-pi:~/encrypted_chat_client$ nano secret.key
lukes@lab-pi:~/encrypted_chat_client$ source venv/bin/activate
(venv) lukes@lab-pi:~/encrypted_chat_client$ python client.py
Enter server ZeroTier IP: 10.178.192.58
Enter nickname: Luke
Type /quit to exit the chat.
> Luke joined the chat!
Hi
> Luke: Hi
hi



this is luke
> Luke: this is luke
hi
```

```
ubuntu@Sebas:~$ python3 -version
python3: can't open file '/home/ubuntu/-version': [Errno 2] No such file or directory
ubuntu@Sebas:~$ python3 -m pip install -user -break-system-packages cryptography
error: externally-managed-environment

× This environment is externally managed
╰─> To install Python packages system-wide, try apt install
    python3-xyz, where xyz is the package you are trying to
    install.

    If you wish to install a non-Debian-packaged Python package,
    create a virtual environment using python3 -m venv path/to/venv.
    Then use path/to/venv/bin/python and path/to/venv/bin/pip. Make
    sure you have python3-full installed.

    If you wish to install a non-Debian packaged Python application,
    it may be easiest to use pipx install xyz, which will manage a
    virtual environment for you. Make sure you have pipx installed.

    See /usr/share/doc/python3.12/README.venv for more information.

note: If you believe this is a mistake, please contact your Python installation or OS distribution provider. You can override this, at the risk of breaking your Python installation or OS, by passing --break-syste
m-packages.
hint: See PEP 668 for the detailed specification.
ubuntu@Sebas:~$
```

```
64 bytes from 10.178.192.58: icmp_seq=12 ttl=64 time=0.1 ms
64 bytes from 10.178.192.58: icmp_seq=13 ttl=64 time=0.1 ms
64 bytes from 10.178.192.58: icmp_seq=14 ttl=64 time=0.1 ms
64 bytes from 10.178.192.58: icmp_seq=15 ttl=64 time=0.1 ms
64 bytes from 10.178.192.58: icmp_seq=16 ttl=64 time=0.1 ms
64 bytes from 10.178.192.58: icmp_seq=17 ttl=64 time=0.1 ms
^C
--- 10.178.192.58 ping statistics ---
17 packets transmitted, 16 received, 5.88235% packet loss, time 16514ms
rtt min/avg/max/mdev = 0.140/0.165/0.192/0.012 ms
mysecl lukes@Lukes-~/python_packet_chat2 ping 0.0.0.0
PING 0.0.0.0 (0.0.0.0) 56(84) bytes of data.
64 bytes from 10.178.192.58: icmp_seq=1 ttl=64 time=0.1 ms
64 bytes from 10.178.192.58: icmp_seq=2 ttl=64 time=0.1 ms
64 bytes from 10.178.192.58: icmp_seq=3 ttl=64 time=0.1 ms
64 bytes from 10.178.192.58: icmp_seq=4 ttl=64 time=0.1 ms
64 bytes from 10.178.192.58: icmp_seq=5 ttl=64 time=0.1 ms
64 bytes from 10.178.192.58: icmp_seq=6 ttl=64 time=0.1 ms
64 bytes from 10.178.192.58: icmp_seq=7 ttl=64 time=0.1 ms
64 bytes from 10.178.192.58: icmp_seq=8 ttl=64 time=0.1 ms
64 bytes from 10.178.192.58: icmp_seq=9 ttl=64 time=0.1 ms
64 bytes from 10.178.192.58: icmp_seq=10 ttl=64 time=0.1 ms
64 bytes from 10.178.192.58: icmp_seq=11 ttl=64 time=0.1 ms
^C
--- 10.178.192.58 ping statistics ---
11 packets transmitted, 11 received, 0.00909% packet loss, time 10219ms
rtt min/avg/max/mdev = 0.140/0.150/0.163/0.009 ms
mysecl lukes@Lukes-~/python_packet_chat2 nano client.py
Enter server IP: 10.178.192.58
Enter nickname: Lukes
Traceback (most recent call last):
  File "/home/lukes/python_packet_chat2/client.py", line 51, in <module>
    client = Client(host, 5555, nickname)
  File "/home/lukes/python_packet_chat2/client.py", line 21, in __init__
    self.client.connect((self.host, self.port))
    ~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^
TimeoutError: [Errno 110] Connection timed out
mysecl lukes@Lukes-~/python_packet_chat2 nano encryption.py
mysecl lukes@Lukes-~/python_packet_chat2 ls
encryption.py  client.py  secret.key  server.py
mysecl lukes@Lukes-~/python_packet_chat2 nano server.py
mysecl lukes@Lukes-~/python_packet_chat2 nano client.py
mysecl lukes@Lukes-~/python_packet_chat2 ls
encryption.py  client.py  secret.key  server.py
mysecl lukes@Lukes-~/python_packet_chat2 python -m py_compile encryption.py server.py client.py
mysecl lukes@Lukes-~/python_packet_chat2 python server.py
[SERVER] Listening...
[LISTENING] Server running on 0.0.0.0:5555

[SERVER MESSAGE] Connected with ('127.0.0.1', 35270)
[SERVER MESSAGE] Connected with ('127.0.0.1', 35272)
mysecl lukes@Lukes-~/python_packet_chat2 python client.py
Enter server IP: 10.178.192.58
Enter nickname: Lukes
Type /quit to exit the chat.
hi
hi

Hi
Hi

Hi
Hi
Hi
Hi
Hi
Hi
Hi
Hi
Hi
Luke joined the chat!
Hi
Hi
hi
hi


Luke: this is luke

hi
hi
```

Troubleshooting (Basic)

Problem: Installing required libraries

What didn't work:

pip3 install cryptography

pip3 install –upgrade pip setuptools wheel

sudo apt install –reinstall python3-pip

What did work:

sudo apt install build-essential libssl-dev python3-dev

sudo apt install –reinstall python3-pip

python3 -m pip install –user cryptography

python3 -c "import cryptography; print(cryptography__version__)"

Problem: No connection between Pis even with ZeroTier Network

Solution (Microsoft Copilot): Server side needed to allow port 5555 through UFW and then connection was possible.

5️⃣ Firewall or security software is blocking port 5555

This is rare on ZeroTier, but possible if someone changed settings.

Test from a client:

Code
```
nc -vz <server-zerotier-ip> 5555
```

Expected:

Code
```
succeeded!
```

If you get:

Code
```
Connection refused
```

→ The server is reachable, but nothing is listening.

If you get:

Code
Connection timed out

→ Something is blocking the port.

3️⃣ Port 5555 is not open because the server isn't listening

Even if ZeroTier works, if nothing is listening on port 5555, clients can't connect.

On the server Pi, run:

Code
```
sudo netstat -tulpn | grep 5555
```

You should see:

Code
```
tcp 0 0 0.0.0.0:5555 0.0.0.0:* LISTEN python3
```

If you see nothing → the server is not running.

```
sudo netstat -tulpn | grep 5555
```

```
ss -tulpn | grep 5555
```

```
sudo ufw status
sudo ufw allow 5555/tcp
```

```
ip a
sudo zerotier-cli listnetworks
```

Milestone 4 – Security Demonstration through MiTM/Wireshark (plus some troubleshooting)

Copilot Assisted Walk-Through:

What the capture shows at a glance

A typical capture contains three kinds of evidence:

- **TCP session setup** — SYN, SYN/ACK, ACK between client and server
- **Encrypted application data** — Fernet ciphertext inside TCP payloads
- **Metadata leakage** — IPs, ports, packet sizes, timing, but no readable content

This is the core of the MiTM demonstration: visibility of transport, invisibility of content.

What the attacker sees in Wireshark

A realistic capture includes:

1. Connection establishment

You'll see packets like:

- `Client → Server: SYN`
- `Server → Client: SYN/ACK`
- `Client → Server: ACK`

This proves the attacker can observe the handshake and identify the communicating hosts.

2. Encrypted chat messages

Each message appears as:

- A TCP segment with a payload of random-looking bytes
- No readable text
- No way to reconstruct the original message

The payload typically looks like:

Code
gAAAAABl... (Fernet ciphertext)

Wireshark will label it as **Application Data** or **Encrypted Data**, depending on the interface.

3. "Follow TCP Stream" shows gibberish

When you right-click → Follow → TCP Stream, the attacker sees:

Code
....gAAAAABlY...

This is the strongest visual proof for your report: the attacker can capture everything but cannot read anything.

What the attacker cannot see

Even though the attacker sees every packet, they cannot see:

- Nicknames
- Chat messages
- Commands
- Server broadcasts
- Any plaintext content

Fernet encryption ensures confidentiality even when the attacker has full packet visibility.

What the simulation looks like step-by-step

A clean demonstration typically follows this sequence:

1. **Start the server** on one Pi.
2. **Start two clients** on other Pis.
3. **Begin a Wireshark capture** on the attacker Pi (or your laptop).

Filter traffic using:
Code
tcp.port == 5555

4.
5. **Send messages** between clients.
6. **Observe encrypted payloads** in Wireshark.
7. **Follow TCP Stream** to show unreadable ciphertext.
8. **Screenshot** the encrypted blobs for your report.

This produces a clear, academically strong MiTM demonstration.

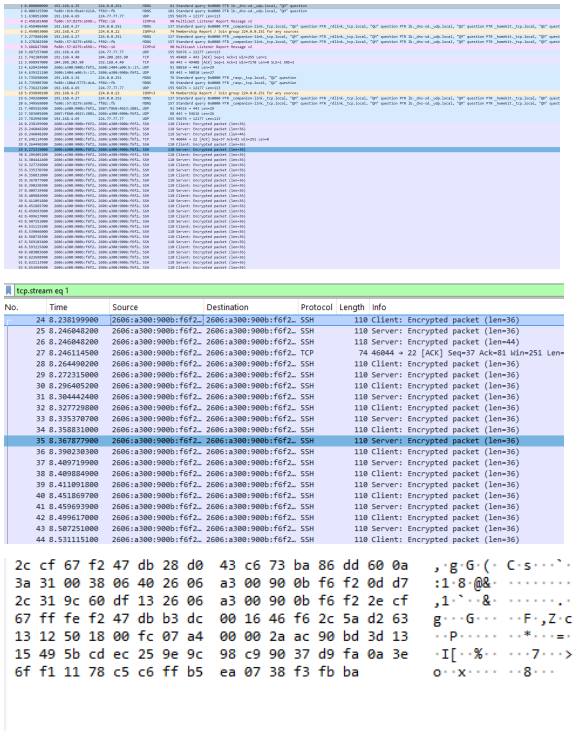
What a screenshot typically contains

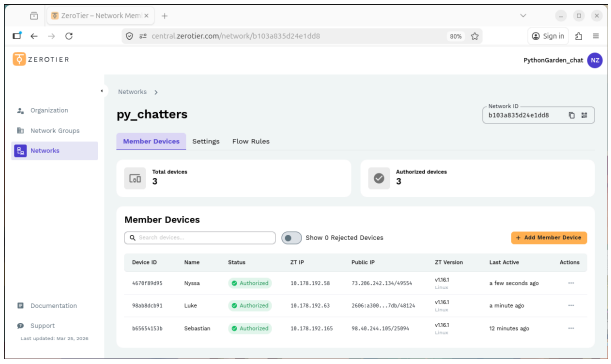
A good screenshot for your report usually shows:

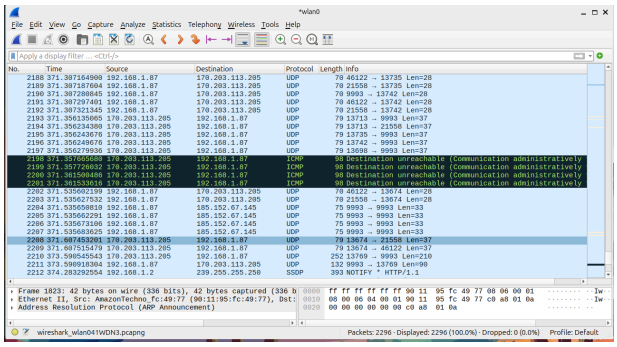
- Source IP (client Pi)
- Destination IP (server Pi)

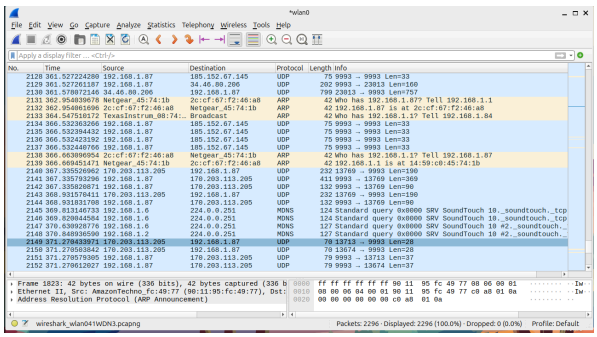
- TCP port 5555
- Packet length
- Encrypted payload bytes
- “Follow TCP Stream” window full of unreadable ciphertext

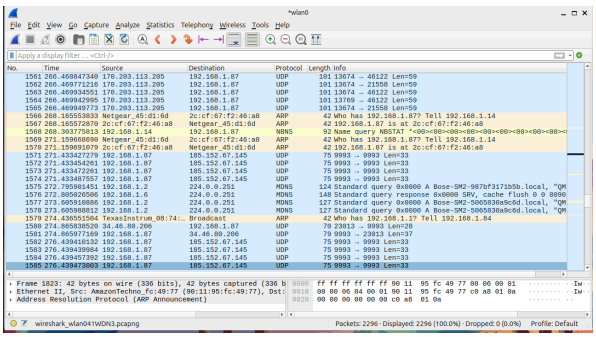
It visually proves that the attacker can intercept but cannot decrypt.











From the screenshots, I can see you’re capturing traffic in **Wireshark** on `wlan0`, and the packets shown are mostly:

- UDP **9993**
- UDP **137xx**
- ARP / MDNS
- ICMP “Destination unreachable”

But **no traffic involving port 5555** appears. Let’s break down the likely reasons. 🔎

1️⃣ Your capture interface is the main clue

Your Wireshark window shows:

*wlan0

But your **ZeroTier** network uses a **virtual interface** (usually called):

ztxxxxxxxx

Example:

zt0
zt3jn2k8l
ztabcdefg

Traffic between ZeroTier peers (like your 10.178.192.x addresses) **does NOT pass through wlan0 directly**. It goes through that virtual interface.

So Wireshark on `wlan0` only sees:

- encrypted ZeroTier transport traffic
- NAT traversal packets (like UDP 9993)

That's exactly what your capture shows.

2️⃣ Why you see UDP 9993

Port **9993** is the default port used by **ZeroTier** for encrypted peer communication.

So what you're seeing is:

192.168.1.87 → 170.203.113.205 UDP 9993

This is **ZeroTier tunnel traffic**, not your application traffic.

Your **port 5555 is inside that encrypted tunnel**, so Wireshark on wlan0 cannot see it.

3️⃣ To see port 5555 you must capture on the ZeroTier interface

Run:

```
ip a
```

Look for something like:

```
ztxxxxxxxx
```

Then start Wireshark on that interface:

```
sudo wireshark
```

Capture on:

```
ztxxxxxxxx
```

Now apply filter:

```
tcp.port == 5555
```

or

```
udp.port == 5555
```

Then you should see the traffic.

4️⃣ Quick test to confirm

From one node:

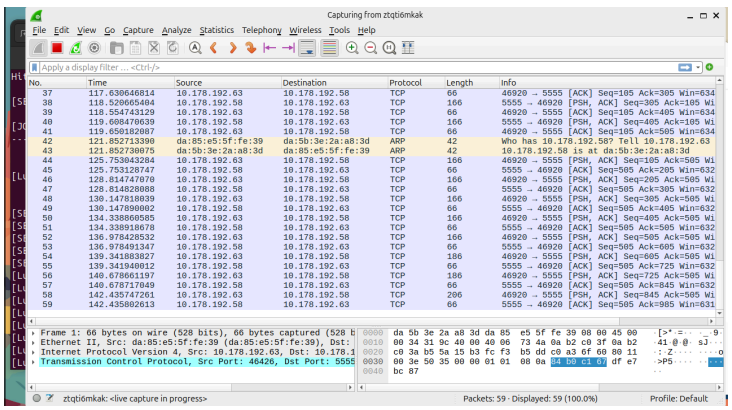
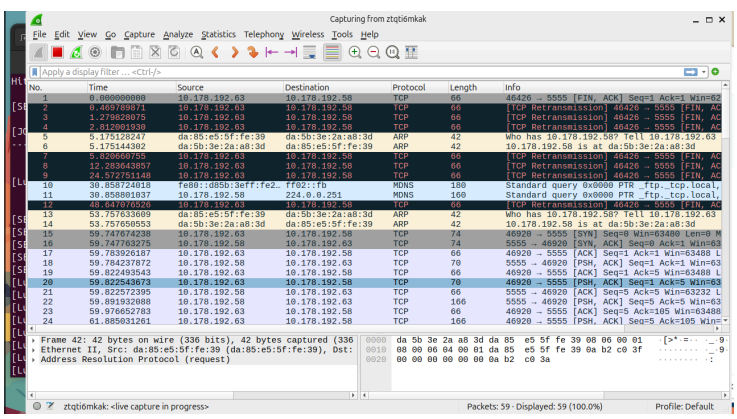
```
nc -l 5555
```

From another node (using ZT IP):

```
nc 10.178.192.58 5555
```

Capture on the **zt interface** and you should immediately see port 5555 packets.

✅ Summary

You don't see port **5555** because:

- You're capturing on **wlan0**
- Your app traffic goes through the **ZeroTier virtual interface**
- On wlan0 it appears only as **encrypted UDP 9993**